PROGRAMMING & CS FUNDAMENTALS

Git Internals Visual Guide

Blobs, trees, commits, and the DAG — what actually happens under git's hood.

ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

Objects: Blob, Tree, Commit

Everything is a content-addressed object in .git/objects.

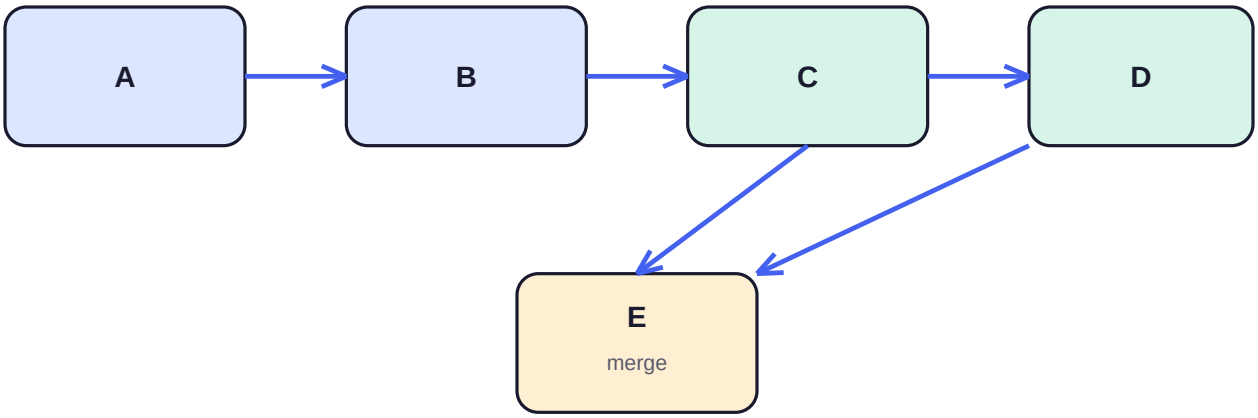
```
$ git cat-file -p HEAD        # unpack any object by sha
$ git hash-object file.txt    # see the blob sha yourself
# identical content -> identical sha -> stored ONCE (dedup!)
```

- Content addressing: the sha IS the identity — change one byte, new object.
- Trees nest: a tree entry can point to a blob (file) or another tree (subdir).
- Tags are objects too (annotated); branches are just pointers (next step).

STEP 2/6

The Commit DAG

History is a directed acyclic graph of commits.



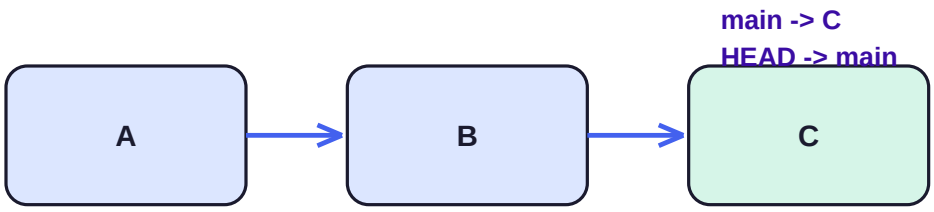
A <- B <- C <- D linear; E has TWO parents (C and D) — a merge commit.

- Each commit points to its parent(s) — history is immutable, only refs move.
- 'Reachable from X' = ancestors of X — that's what log/fetch/push reason about.
- DAG, not a tree: merges give nodes two parents; that's the whole game.

STEP 3/6

Refs: Branches, Tags, HEAD

Branches are sticky notes on commits — 41 bytes that move.



commit -m: new commit D, main slides forward to D. That's ALL a commit does to branches.

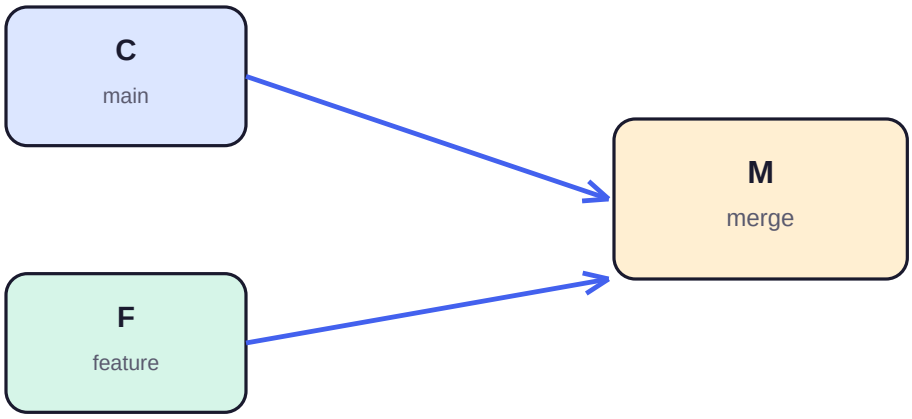
```
$ cat .git/refs/heads/main      # literally a 40-char sha
$ git symbolic-ref HEAD         # refs/heads/main
# detached HEAD: HEAD points at a sha directly — commits can strand
```

This mental model dissolves 90% of git confusion: commits are frozen, refs are movable labels. Every command just moves labels or makes commits.

STEP 4/6

Merge: Uniting Histories

Two parents, one commit — preserving the true story.



M's tree = combined result; parents = (C, F). History shows the branch REALLY happened.

```
$ git merge feature           # fast-forward if linear, else merge commit
# conflicts: BOTH sides edited same lines — you resolve, then commit
# octopus: merge 3+ heads at once (rare, satisfying)
```

- Fast-forward: no new commit when one side is strictly ahead — just slide the ref.
- Merge commits are documentation: 'feature X landed here, with this context'.

STEP 5/6

Rebase: Rewriting Onto

Replay your commits on a new base — linear history, rewritten shas.

before: main: A-B-C feature: A-B-X-Y

after rebase: main: A-B-C feature: A-B-C-X'-Y' (COPIES!)

```
$ git rebase main              # on feature: replay X,Y onto C
# X' has new sha (new parent!) — old X,Y still exist until GC
# GOLDEN RULE: never rebase commits others have pulled
```

- Rebase = clean linear story; merge = honest branching story — teams pick one.
- Interactive rebase (-i): squash, reword, reorder, drop — history as an editable draft.
- Rewriting public history strands collaborators — force-push only your own branches.

STEP 6/6

Reflog: The Safety Net

Every ref move is logged — nothing is truly lost for ~90 days.

```
$ git reflog                  # HEAD@{0}, HEAD@{1}, ... every move
$ git reset --hard HEAD@{2}   # undo that bad rebase/merge/reset
$ git fsck --lost-found       # dangling commits: rescue mission
```

- 'I deleted my branch!': the commits live on — reflog finds the sha, re-create the ref.
- reset --soft/--mixed/--hard: move HEAD (and optionally index/worktree) — know which.
- GC prunes unreachable objects after ~2 weeks (reflog expiry ~90 days) — rescue promptly.
- The index (staging area) is a state between worktree and HEAD — commit snapshots the index.

STEP ✓/✓

Key Takeaways

Git Internals — Remember

- Objects: blobs (content), trees (directories), commits (snapshot + parents) — all content-addressed.
- History is a DAG; branches/tags are movable refs, HEAD points at your current ref.
- Merge preserves branching truth (two parents); rebase replays as clean linear copies — never rebase public history.
- Fast-forward slides refs with no new commit; conflicts mean both sides touched the same lines.
- Reflog logs every ref move — deleted branches and bad rebases are recoverable.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev